Cindy and Her 3 Goons' Design Doc

P05 – *Le Fin*

PROJECT NAME: *Snorphan*

TARGET SHIP DATE: 2026-06-01

ROSTER

Name	Email	Primary Role	Secondary Role
Carrie Ko	carriek3@nycstudents.net	PM	Mediator
Cindy Liu	cindyl125@nycstudents.net	Devo	Physics and Movement
Joyce Lin	joycel78@nycstudents.net	Devo	Interactions
Jeff Ou	jiefengo@nycstudents.net	Devo	Server Handling

PROJECT SUMMARY

Snorphan is an open world puzzle-adventure game. You're a lonely orphan who has no friends and in order to cure your insatiable loneliness, you spend your life building snowmen. But, oh no! Your snowmen are all sad and naked! Thus, as a last resort, you finally decide to socialize and look to the town's villagers for help. And boy do they have some work for you to do.

Problem Being Solved

We are bringing awareness to the loneliness of orphans and snowmen.

Target Users

- Orphan lovers
- Snowmen lovers

Why This Project Matters

This is a very real issue in the real world. Orphanages have low adoption rates and orphans who get left behind lose their most important human connections. This game is meant to give awareness to the love that orphans deserve.

MINIMUM VIABLE PRODUCT (MVP) SCOPE

Core Features

We will have a working story game with an orphan character that can make snowmen, obtain items, and interact with NPCs (dialogue, quests, etc). There will be a fixed terrain and map.

Stretch Features

2) Using items (ie. throwing snowballs, using a shovel, etc.).
3) Fighting a boss.

Explicit Non-Goals

Features intentionally **excluded**
* Fight your snowmen
* Selecting different base characters

TECHNOLOGY STACK

Layer	Selected Tool
Backend Framework	Node.js and Flask
Frontend Framework	none
Database	SQLite

Layer	Selected Tool
Authentication	Flask Sessions
ORM / DB Library	none

Why This Stack Was Chosen

- `Backend Framework`: Node allows for easy set up of WebSocket servers as it supports it natively as opposed to Flask. This makes it more convenient when attempting to set up multiplayer features (though main story quests are player specific)! However, using Flask in addition to Node makes it easier to route our pages as well as manage SQLite3 interactions without extra research!!
- `Frontend Framework`: We will not be utilizing a Frontend Framework as we prefer manually doing CSS :D
- `Database`: As most members of our group are more accustomed to SQLite and there is no particular reason why we would need document-oriented databasing
- `Authentication`: We will be utilizing Flask sessions as we have always done!

TEAM OWNERSHIP PLAN

Each member must own meaningful deliverables

Team Member	Primary Ownership	Secondary Ownership	Specific Deliverables
Carrie Ko	Visuals	Backend Development	Character models, SQLite3 databases, Story Quests
Cindy Liu	Backend Development	Visuals	Authentication system, Movement system
Joyce Lin	Visuals	Backend Development	Map/terrain, Model rigging, Story Quests
Jeff Ou	Backend Development	Visuals	Node WebSocket Server, Multiplayer connections

Program Components + Explanation

Backend

- `SQLITE`: db storage for all our storage needs
- `__init.py__`: serves our html pages and declares all dbs and helper functions

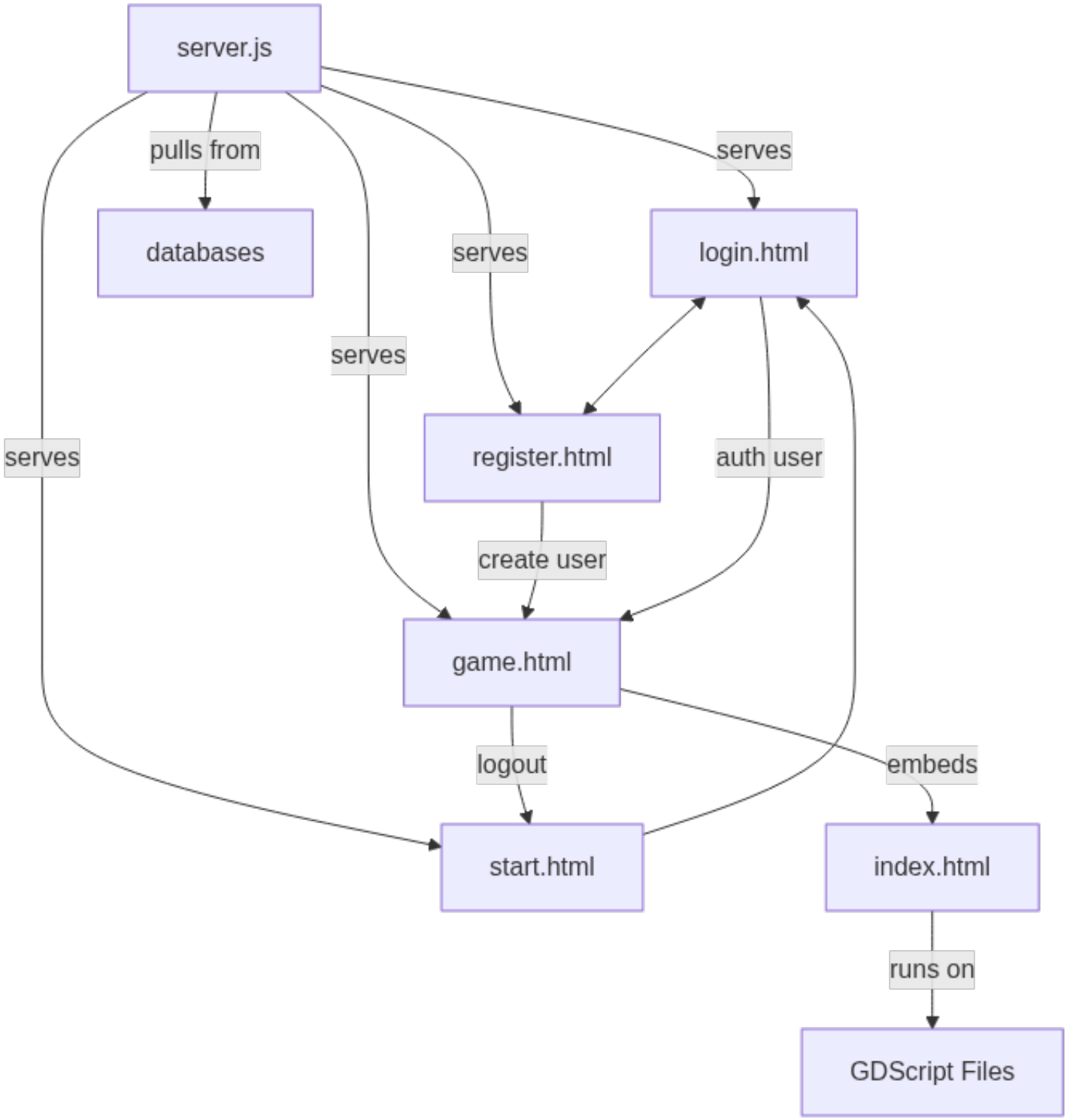
HTML (frontend display)

- `start.html`: startscreen for all your starts
- `exit.html`: exit for all your exits (logs you out)
- `credit.html`: we get recognition (yay!)
- `login.html`: you can login as a user wow amazing 10/10
- `register.html`: wow you can register up and actually play our game wow amazing 11/10
- `index.html`: contains exported Godot app and allows for embedding
- `game.html`: this is where our game will live and where we will serve it for you to see

Javascript

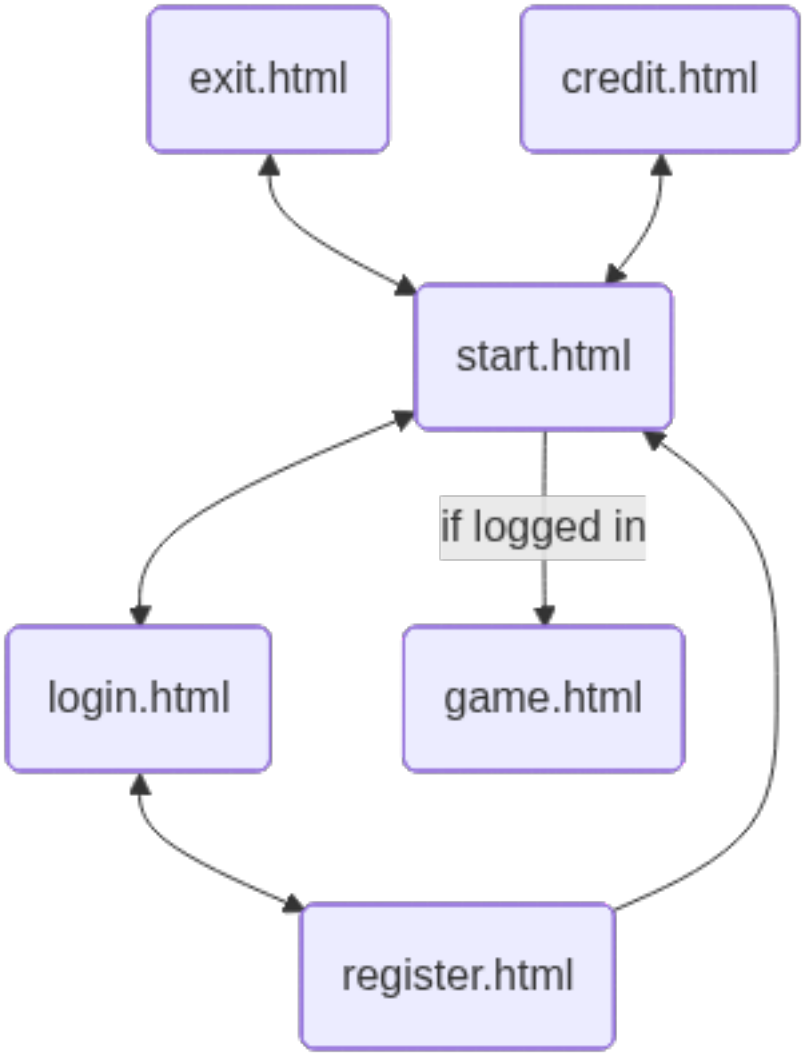
- `server.js`: node setup code

Component Map



Note: GDScript files are Godot-specific files that allows interactions and events within the app to occur. Javascript within Index.html contains the exported engine object which allows the app to run. GDScript files are only relevant within Godot itself but since Index.html is rendering the app itself, it technically runs on these script files as well.

Site Map (front end)



Key User Stories

Orphano McLover

As an orphan lover, I want to be able to properly experience life as an orphan making snowmen so that I can understand the life and mindset of orphans better.

Snowmin Loverman

As a snowman lover, I want to be able to make lots of snowmen so that I can satisfy my cravings for snowmen even when it isn't winter.

Database Organization

USER

TYPE	VALUE	ADDITIONAL SPECIFICATIONS
TEXT	username	PK
TEXT	password	NOT NULL
INTEGER	hp	NOT NULL
INTEGER	stamina	NOT NULL
TEXT	item1	FK
TEXT	item2	FK
TEXT	item3	FK

TYPE	VALUE	ADDITIONAL SPECIFICATIONS
TEXT	item4	FK
TEXT	item5	FK
TEXT	item6	FK
INTEGER	item1Count	
INTEGER	item2Count	
INTEGER	item3Count	
INTEGER	item4Count	
INTEGER	item5Count	
INTEGER	item6Count	

ITEM

TYPE	VALUE	ADDITIONAL SPECIFICATIONS
TEXT	name	PK
TEXT	desc	NOT NULL
TEXT	image	NOT NULL
INTEGER	maxCount	NOT NULL

ENCYCLOPEDIA

TYPE	VALUE	ADDITIONAL SPECIFICATIONS
TEXT	item	FK
TEXT	userThatFound	FK

SETTINGS

TYPE	VALUE	ADDITIONAL SPECIFICATIONS
TEXT	setting	PK
TEXT	user	FK
BOOLEAN	boolValue	
INTEGER	intValue	

NPC

TYPE	VALUE	ADDITIONAL SPECIFICATIONS
TEXT	name	PK
TEXT	dialogue	NOT NULL
TEXT	completedQuestUsers	

- Dialogue EX. ~ "DG1$A.OPTION$1B.OPTION2&A.DG2...&B.DG2..."
  - Dialogues will be split with & while the options themselves are split with $ (arr[0] will always point to the printed dialogue)
  - Each line of dialogue will be preceded by the letter corresponding to each possible option
- completedQuestUsers EX. ~ "USER1$USER$2USER3$USER4"
  - Used to check if user has completed the quest!

TESTING PLAN

Upon completion of each game component (assignation of story quests, NPC interactions, snowman rolling, etc.), they will be tested through gameplay as well as cheats designated for debugging. Any associated features will not be worked on until the basic component is working!

Components such as backend server handling will be debugged as it always has been, through debug statements and pair handling.

TIMELINE

WEEK 1:

- Inserted models (* Models will be acquired through the internet but rigging will be done through blender)
- Working movement system
- Terrain created
- Server connections and multiplayer set up
- Exit to startscreen and log-out system

WEEK 2:

- Inventory system
- Snowman creation system
- NPC interaction (dialogue system)
- Cave systems created (with purpose!) — MAY BE CUT
- Puzzles set up and planned out
- Item usage system
- Item encyclopedia

WEEK 3:

- Story quest system set up
- Cross player interaction (snowball throwing)
- Puzzles all implemented
- Final boss/disaster sequence implemented
- animations

INTERNAL DEADLINES:

1) `Working movement system` ~ 5/12
2) `Working node websocket server up on primary domain` ~ 5/12
3) `multiplayer system connections` ~ 5/13
4) `character models and map` ~ 5/15
5) `inventory system` ~ 5/18
6) `dialogue system` ~ 5/19
7) `puzzles set up` ~ 5/20
8) `snowman creation system` ~ 5/21
9) `item usage system and encyclopedia` ~ 5/22
10) `story quests set up` ~ 5/26
11) `puzzles all implemented` ~ 5/26
12) `final boss/disaster completion` ~ 5/26
13) `cross player interaction` ~ 5/30 (Not the most important feature!)

COMPLETION CRITERIA

Project is considered complete when all of the following are true: 1) the app runs without crashing, ever 2) start/login/register routes into the game, somehow, in reasonable order 3) promised snowman functionality exists and works 4) all assets load in on the average runthrough 5) you cannot get stuck in-game (bugged progression, model stuck underground, etc.) w/o logout/restart available

OPEN QUESTIONS

Will the final challenge be a boss fight (big yeti or snowman?!) or will it be an escape sequence (avalanche, collapsing cave, etc)?

APPENDIX

Empty! For Now!